



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

**PERSONAL FINANCE MANAGEMENT AND BUDGET
ANALYZER USING C++**

A CAPSTONE PROJECT REPORT

*A Capstone Project Report submitted in partial fulfilment of the requirements
for the Course of*

DSA0113 - OBJECT ORIENTED PROGRAMMING WITH C++

to the award of the degree of

**BACHELOR OF ENGINEERING / BACHELOR OF TECHNOLOGY IN
COMPUTER SCIENCE AND ENGINEERING**

Submitted by

Prena M 192524056

Under the Supervision of

Dr. Sivanantham Sivakumar

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

SEPTEMBER 2026

SIMATS ENGINEERING
Saveetha Institute of Medical and Technical Sciences
Chennai-602105

BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled "Personal Finance Management and Budget Analyzer Using C++" has been carried out by **PrenaM (192524056)** under the supervision of **Dr.Sivanantham Sivakumar** and is submitted in partial fulfilment of the requirements for the current semester of the B. Tech Computer Science and Engineering programme at Saveetha Institute of Medical and Technical Sciences, Chennai.

COURSE COORDINATOR

Dr. C. Sivasankar
Professor
Department of CSE,
Saveetha School of Engineering,
SIMATS, Chennai

COURSE FACULTY

Dr. Sivanantham
Professor
Department of CSE,
Saveetha School of Engineering,
SIMATS, Chennai

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

DECLARATION

I am **Prena M[192524056]** of the CSE, SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai, hereby declare that the Capstone Project Work entitled "**Personal Finance Management and Budget Analyzer Using C++**" is the result of our own bonafide efforts. To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with the principles of engineering ethics and academic integrity. All sources, references, data, and other materials used in the project have been appropriately acknowledged. We further declare that the data, results, analysis, and findings presented in this report have not been fabricated, falsified, or manipulated. Any external tools, software, datasets, computational resources, or AI-assisted tools used during the project have been appropriately disclosed. We confirm that all applicable ethical, academic, institutional, and professional requirements have been duly followed throughout the planning, execution, analysis, and documentation of the project.

Place:

Date:

Signature of the Student with Name

PRENA M[192524056]

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

Individual Contribution Statement

(Mandatory for all team projects)

Student Name / Reg. No.	Specific Responsibilities	Design & Development Contribution	Testing & Analysis Contribution	Report Contribution	Approx. Contribution (%)
Prena M [192524056]	Module 1 -- Income & Expense Management (data structures, CRUD operations)	Designed Transaction class and file- handling logic	Unit-tested add/edit/delete transaction functions	Chapters 1, 4 (4.2)	[33%]
Deepika NS [192421348]	Module 2 -- Budget Analysis, Tracking & What-If Simulation	Designed budget- comparison and simulation algorithms	Tested budget alerts and simulation scenarios	Chapters 2, 3	[33 %]
Soundarya [192524342]	Module 3 -- Savings & Financial Report Generation	Designed report- generation and savings- goal logic	Validated report accuracy against sample data	Chapters 4, 5	[33%]

Total: 100%

ABSTRACT

Effective personal financial management is a growing necessity as individuals increasingly rely on manual spreadsheets or fragmented mobile applications to track their income, spending, and savings, often without a clear picture of their overall financial health. This project, titled “Personal Finance Management and Budget Analyzer Using C++,” presents a lightweight, offline, console-based financial management system implemented entirely in C++ using core data structures and file handling, without dependence on external databases or internet connectivity. The system is organised into three functional modules. The Income & Expense Management module allows users to record, categorise, edit, and delete income and expense transactions, storing them persistently using structured file I/O. The Budget Analysis, Tracking & What-If Simulation module compares actual spending against user-defined category-wise budgets, flags overspending, and allows the user to simulate hypothetical changes in income or expenses to observe their projected impact on the monthly budget before committing to a decision. The Savings & Financial Report module computes net savings, tracks progress toward user-defined savings goals, and generates a consolidated financial summary report. The system uses arrays/vectors, structures, and file streams to maintain transaction records, and applies simple statistical computation (totals, averages, percentage variance) to support budget analysis. Testing against representative transaction datasets shows that the system accurately classifies spending, correctly flags budget overruns, and produces consistent financial summaries. The project demonstrates that a well-structured, algorithmically sound C++ application can provide meaningful budgeting assistance without the overhead of a full database or GUI framework, and can serve as a foundation for future extensions such as graphical interfaces, database integration, or predictive expense forecasting.

Keywords: Personal Finance Management, Budget Analysis, C++, File Handling, What-If Simulation, Expense Tracking, Financial Report Generation

TABLE OF CONTENTS

Sl. No.	Title	Page No.
i	CERTIFICATE FROM THE SUPERVISOR	ii
ii	DECLARATION BY THE CANDIDATE	iii
iii	INDIVIDUAL CONTRIBUTION STATEMENT	iv
iv	ABSTRACT	v
v	LIST OF FIGURES	vi
vi	LIST OF TABLES	vii
vii	LIST OF ABBREVIATIONS	viii
1	CHAPTER 1: Introduction and Engineering Problem	
2	CHAPTER 2: Literature Review and Existing Solutions	
3	CHAPTER 3: Engineering Design & Trade-offs, Sustainability, Ethics, Safety, Risk & Security	
4	CHAPTER 4: Implementation, Testing & Results, Project Management	
5	CHAPTER 5: Conclusion, Future Work and Reflection	
	References	
	Appendices	

LIST OF FIGURES

Figure No.	Figure Name	Page No.
Fig. 1.1	Overview of the Personal Finance Management System	
Fig. 3.1	System Architecture Diagram	
Fig. 3.2	Module Interaction / Data Flow Diagram	
Fig. 4.1	Main Menu Flowchart	
Fig. 4.2	Sample Console Output -- Transaction Entry	
Fig. 4.3	Sample Console Output -- Budget Analysis & What-If Simulation	
Fig. 4.4	Sample Console Output -- Savings & Financial Report	
Fig. 4.5	Project Gantt Chart / Milestone Timeline	

LIST OF TABLES

Table No.	Table Name	Page No.
Table 2.1	Comparative Analysis of Existing Personal Finance Tools	
Table 3.1	Stakeholder / User Requirements	
Table 3.2	Functional & Non-Functional Requirements	
Table 3.3	Constraints & Applicable Standards	
Table 3.4	Design Specifications	
Table 3.5	Alternative Solutions & Evaluation	
Table 3.6	Trade-off Analysis	
Table 3.7	Sustainability, Ethics, Safety, Risk & Security	
Table 3.8	Risk Register	
Table 4.1	Tools / Software Used	
Table 4.2	Test Plan & Verification	
Table 4.3	Comparison with Existing Solutions & Achievement of Objectives	
Table 4.4	Project Roles & Contribution	
Table 4.5	Estimated vs Actual Cost / Effort	

LIST OF ABBREVIATIONS / SYMBOLS

Symbol	Description	Unit
CLI	Command Line Interface	--
STL	Standard Template Library	--
CSV	Comma Separated Values	--
I/O	Input / Output	--
OOP	Object-Oriented Programming	--
Rs.	Indian Rupees	₹
% Var	Percentage Variance (Actual vs Budgeted)	%
LOC	Lines of Code	--

CHAPTER 1

INTRODUCTION AND ENGINEERING PROBLEM

1.1 Background

Personal financial management involves tracking income, controlling expenditure, planning budgets, and building savings over time. Traditionally, individuals manage this using pen-and-paper ledgers, spreadsheets, or third-party mobile applications. While spreadsheets are flexible, they require manual formula maintenance and are error-prone; commercial mobile finance apps, on the other hand, often require internet connectivity, account linking, and share user financial data with external servers, which raises privacy concerns. C++ provides a fast, resource-efficient environment with strong support for structures, arrays/vectors, and file handling, making it well suited for building a self-contained, offline personal finance tool that runs entirely on the user's own machine.

1.2 Need for the Project

The need for this project arises from the following observations:

- Why the problem needs to be addressed: Individuals frequently overspend because they lack real-time visibility into how actual spending compares against a planned budget.
- Who is affected: Students, young professionals, and small households who do not use paid financial software are most affected, as they typically rely on ad-hoc tracking methods.
- Existing limitations: Spreadsheet-based tracking demands manual updates and offers no automated alerts; many mobile apps require continuous internet access, account permissions, and subscription fees.
- Engineering significance: The project applies core data structures (arrays/vectors, structures), file persistence, and simple algorithmic analysis (aggregation, variance, simulation) to solve a practical, everyday engineering problem within a constrained, offline environment.

1.3 Problem Statement

There is a need for a lightweight, offline, C++-based console application that allows a user to record income and expense transactions, compare actual spending against category-wise budgets, simulate the effect of hypothetical financial changes (what-if analysis), and generate a consolidated savings and financial summary report -- all while managing multiple interacting factors such as transaction categories, monthly budget limits, variable income, and persistent local storage, under the realistic constraints of limited memory use, no external database dependency, and simple text-based/CSV file storage.

1.4 Project Aim

To design and develop a modular, C++-based Personal Finance Management and Budget Analyzer that enables users to record income/expenses, analyse and track budgets with what-if simulation, and generate savings and financial reports, using efficient data structures and file handling.

1.5 Project Objectives

1. To design a suitable data structure (Transaction record) to represent income and expense entries with category, amount, date, and type.
2. To develop an Income & Expense Management module supporting add, view, edit, delete, and persistent file-based storage of transactions.
3. To model a Budget Analysis & Tracking module that compares actual category-wise spending against user-defined budget limits.
4. To implement a What-If Simulation feature that projects the effect of hypothetical income/expense changes on the monthly budget without altering saved data.
5. To develop a Savings & Financial Report module that computes net savings, tracks progress toward savings goals, and generates a summary report.
6. To test the system using representative transaction datasets and evaluate accuracy, correctness of budget alerts, and report consistency.
7. To evaluate the system's usability, performance, and scope for future enhancement (e.g., GUI, database, forecasting).

1.6 Scope

Included:

- Recording and categorising income and expense transactions.
- Category-wise budget definition, tracking, and overspend alerts.
- What-if simulation for hypothetical income/expense changes.
- Savings tracking, goal progress, and consolidated report generation.
- Persistent storage using local text/CSV files.

Excluded:

- Bank account linking, online payment integration, or real-time transaction import.
- Graphical user interface (GUI) -- the system is console-based.
- Multi-user/cloud synchronisation and mobile deployment.

Assumptions:

- The user enters transaction data manually and honestly.
- A single user operates the system on a single local machine at a time.
- Currency is assumed to be Indian Rupees (₹) unless otherwise configured.

Boundary Conditions:

- Maximum number of transactions is limited by available memory / configured array-vector capacity.
- Budget periods are assumed to be monthly.

1.7 Expected Engineering Outcome

The expected outcome is a working C++ console-based software system (prototype/application) comprising three integrated modules -- Income & Expense Management, Budget Analysis & Tracking with What-If Simulation, and Savings & Financial Report -- backed by a file-based persistence layer, capable of producing accurate, real-time budget feedback and a consolidated financial report for the end user.

CHAPTER 2

LITERATURE REVIEW AND EXISTING SOLUTIONS

2.1 Review Method

Relevant literature and existing tools were identified through a review of (i) widely used personal finance and budgeting applications (e.g., spreadsheet-based budget templates, Mint-style expense trackers, YNAB-style envelope budgeting tools), (ii) academic project reports and papers on expense-tracking and budgeting systems implemented using C/C++ and file handling, and (iii) standard C++ references on data structures, STL containers, and file I/O. Sources were compared on functionality, technology used, offline capability, and analytical depth.

2.2 Review of Existing Work

Spreadsheet-based budgeting (e.g., Excel/Google Sheets templates) allows manual entry of income and expenses with formula-driven totals, but requires the user to maintain formulas correctly and offers no built-in alerting or simulation capability. Commercial mobile applications such as Mint and YNAB (You Need A Budget) provide automated bank-feed categorisation and rich visual dashboards, but depend on continuous internet connectivity, third-party servers, and often a subscription fee, which raises data-privacy concerns for users who prefer to keep financial data local. Several academic/student projects implement simple C/C++ expense trackers using arrays and file handling; however, most of these focus only on transaction logging and stop short of providing budget-variance analysis or what-if simulation. Standard C++ references on STL vectors, structures, and fstream-based file handling provide the technical foundation this project builds upon for efficient, in-memory transaction management with persistent storage.

2.3 Comparative Analysis

Existing Approach	Advantages	Limitations	Relevance to Proposed Work
Spreadsheet-based budget templates	Familiar, flexible, no coding required	Manual formula maintenance, no alerts, error-prone	Motivates automated, rule-based budget comparison
Mint / cloud-based finance apps	Automated bank-feed sync, rich dashboards	Requires internet, shares data with servers, subscription cost	Motivates a private, offline, local-storage alternative
Basic C/C++ expense-tracker projects	Lightweight, offline, easy to extend	Usually logging-only, no budget analysis or simulation	Provides the technical baseline (arrays/structs/file I/O) extended in this project
YNAB-style envelope budgeting	Strong category-wise discipline, proactive planning	Manual re-allocation overhead, subscription-based	Inspires the category-wise budget & what-if simulation design

2.4 Research / Engineering Gap

Existing lightweight, offline C++-based tools generally support only transaction logging and lack (i) category-wise budget-variance analysis, (ii) an interactive what-if simulation capability to preview the effect of financial decisions before committing them, and (iii) an integrated savings-goal tracker with automated report generation. This leaves a gap for a single, offline, privacy-preserving system that combines all three capabilities using efficient data structures.

2.5 Proposed Contribution

This project addresses the identified gap by integrating three cohesive modules - transaction management, budget analysis with what-if simulation, and savings/report generation -- into a single offline C++ application. Unlike basic logging tools, it applies structured comparison logic (actual vs budgeted, percentage variance) and a non-destructive simulation mode that lets users test hypothetical scenarios without modifying saved records, combining the analytical depth of commercial apps with the privacy and simplicity of a local, dependency-free C++ program.

CHAPTER 3

ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY

3.1 Requirements

Stakeholder / User Requirements

Stakeholder	Requirement
End user (individual / household)	Easily record income and expenses and see where money is going each month
End user	Be alerted when spending in a category exceeds the planned budget
End user	Preview ('what-if') the effect of a planned purchase or income change before it happens
End user	Track progress toward a savings goal and view a consolidated financial report
Academic evaluator	System should demonstrate sound use of data structures, file handling, and modular design

Functional & Non-Functional Requirements

Requirement	Type (Functional/Non-Functional)	Measurable Target
Add / edit / delete income & expense transactions	Functional	Operation completes and persists in < 1 second per transaction
Categorise transactions (e.g., Food, Rent, Travel, Savings)	Functional	Support at least 8 predefined categories
Compare actual vs budgeted spend per category	Functional	Variance computed to 2 decimal-place accuracy
What-if simulation without altering saved data	Functional	Simulation runs on a copy of data; 0 changes to stored file

Requirement	Type (Functional/Non-Functional)	Measurable Target
Generate savings & financial summary report	Functional	Report generated in under 2 seconds for ≤ 1000 records
Data persistence between sessions	Non-Functional (Reliability)	0% data loss across normal program restarts
Usability of console interface	Non-Functional (Usability)	Core task completed within ≤ 5 menu selections
Portability	Non-Functional (Portability)	Compiles with any standard C++17 compiler (g++, MSVC)

3.2 Constraints & Applicable Standards

Standard / Code	Requirement	How Applied in This Project
ISO/IEC 14882 (C++ Language Standard)	Code must conform to standard C++ syntax and semantics	Project developed and compiled under the C++17 standard for portability
Institutional Academic Integrity Policy	Original work with disclosed AI/tool assistance	All AI-assisted drafting is disclosed per the Declaration section
Basic Data-Privacy Good Practice	User financial data should not be transmitted externally without consent	System stores data only in local files; no network transmission is implemented

3.3 Design Specifications

Parameter	Target/Requirement	Unit	Priority
Max transactions supported (in-memory)	≥ 1000	records	High
Transaction entry response time	< 1	second	High
Report generation time (≤ 1000 records)	< 2	seconds	Medium
Budget-variance calculation accuracy	2	decimal places	High
Data file format	CSV / structured text	--	Medium

3.4 Alternative Solutions & Evaluation

Alternative 1: A pure in-memory C++ array/struct-based system with no file persistence -- fast, but all data is lost on program exit.

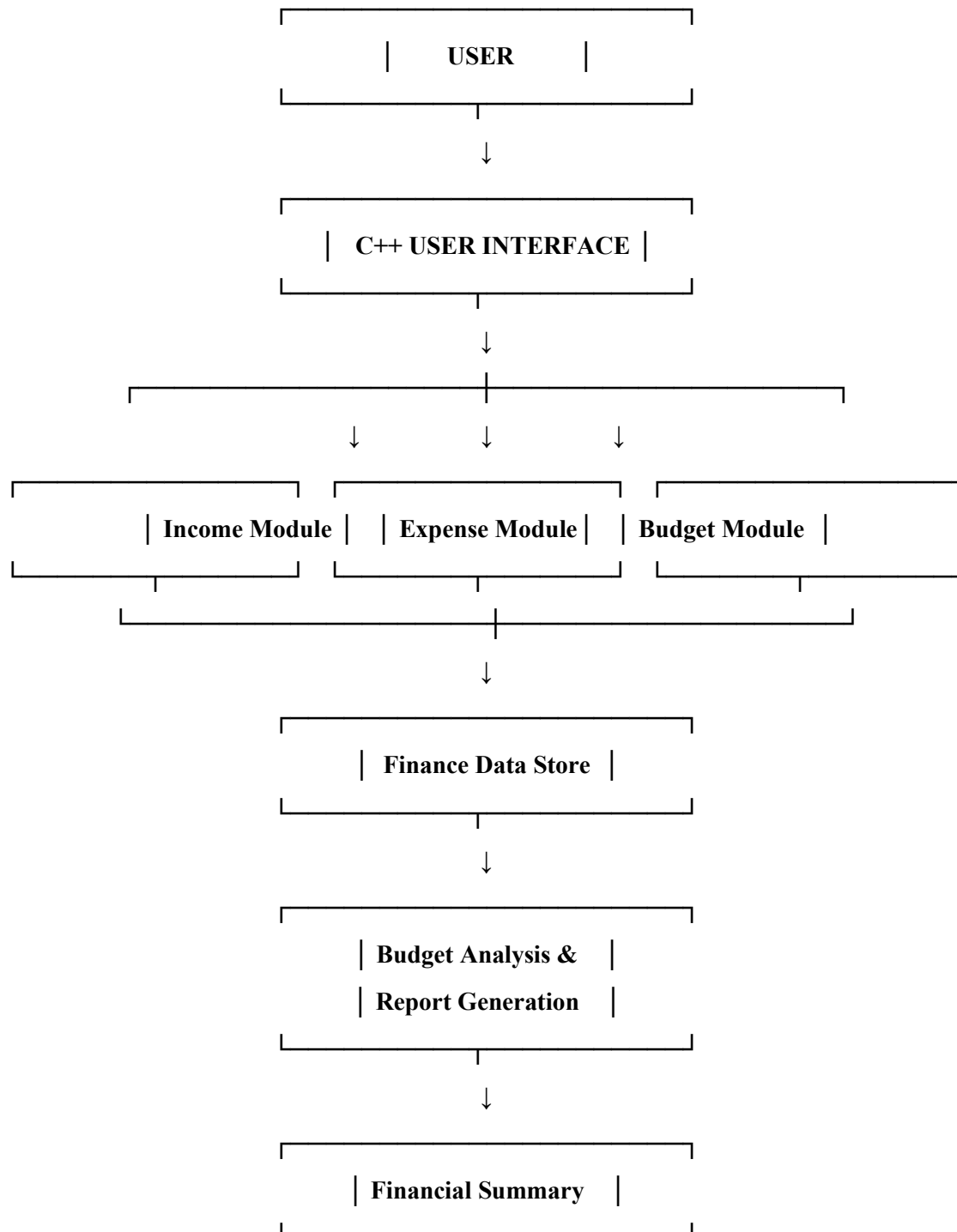
Alternative 2: A C++ system using STL vectors, structures, and fstream-based CSV file persistence -- balances simplicity, speed, and durability of data across sessions.

Alternative 3: A C++ system integrated with a lightweight embedded database (e.g., SQLite) -- offers robust querying but adds external dependency and setup complexity beyond the project scope.

Criterion	Weight	Alternative 1	Alternative 2	Alternative 3
Performance	30%	High	High	Medium
Cost (dev. effort/tools)	20%	Low	Low	Medium-High
Safety (data loss risk)	20%	Low (data lost on exit)	High	High
Sustainability (maintainability)	15%	Low	High	Medium
Reliability	15%	Low	High	High

3.5 Selected Design & Detailed Design

Alternative 2 (STL vectors/structures with fstream-based CSV persistence) was selected as it best satisfies the project's constraints of remaining dependency-free while ensuring data durability across sessions, which Alternative 1 cannot provide, and it avoids the added complexity and setup overhead of an embedded database required by Alternative 3.



Key engineering elements: A Transaction structure holds id, date, category, type (Income/Expense), and amount. A `std::vector<Transaction>` holds all records in memory during a session and is synchronised with a CSV file (`finance_data.csv`) on load/save. The Budget module holds a `std::map<std::string, double>` of category-to-budget-limit, against which aggregated actual spend (computed via a single pass over the transaction vector) is compared to produce a percentage variance per category. The What-If Simulation module operates on a duplicated copy of the transaction/budget data structures so that hypothetical edits never touch the persisted file until the user explicitly confirms.

Systems approach: The Income & Expense module supplies validated transaction records to the Budget Analysis module, which aggregates them by category and period. The Budget module, in turn, feeds computed totals into the Savings & Report module, which combines income, expense, and budget-variance data into a single consolidated report. Each module is implemented as an independent set of functions/classes operating on shared data structures, allowing them to be tested and extended in isolation while remaining interoperable through a common Transaction/Budget data model.

3.6 Trade-off Analysis

Design Decision	Alternative Considered	Benefit	Disadvantage	Final Decision & Justification
Storage mechanism	In-memory only vs CSV file vs SQLite	CSV: durable & dependency-free	CSV: no query language, manual parsing	CSV file chosen -- durability without external dependencies, matches project scope
Simulation approach	Modify live data vs simulate on a copy	Copy: safe, reversible	Copy: extra memory use for duplicate structures	Simulate on a copy -- protects saved data integrity, which outweighs the minor memory overhead
Interface type	Console (CLI) vs GUI	GUI: more user-friendly	GUI: added libraries, longer build time	Console CLI chosen -- fits project timeline and keeps the system lightweight/portable

3.7 Sustainability, Ethics, Safety, Risk & Security

Domain	Consideration	Evidence Evaluated	Impact on Final Decision
Sustainability	Lightweight console app minimises computing resource/energy use compared to always-on cloud finance apps	Program runs locally, no server infrastructure required	Reinforced choice of an offline, local-file architecture
Ethics	User financial data must not be misrepresented or silently altered	What-if simulation clearly separated from saved data to avoid misleading the user	Simulation results are always labelled 'projected' and never overwrite saved records
Health & Safety	Not physically applicable (software-only project); cognitive load considered	Simple menu-driven interface reduces user error	Kept the menu structure short (≤ 5 options per screen)
Security	Local financial data should be protected from accidental loss/corruption	File-write validated with error handling; backup-on-save considered	Added basic file-existence and write-success checks before confirming a save

Risk Register

Risk	Likelihood	Severity	Risk Level	Mitigation	Residual Risk
Data file corruption/loss on abnormal exit	Medium	High	High	Validate writes; maintain a backup copy before overwrite	Low
Invalid user input (e.g., non-numeric amount)	High	Low	Medium	Input validation loops with error prompts	Low
Budget miscalculation due to logic error	Low	High	Medium	Unit testing of aggregation and variance functions against known datasets	Low
Array/vector overflow with very large datasets	Low	Medium	Low	Use dynamic STL containers (std::vector) instead of fixed-size arrays	Low

CHAPTER 4

IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT

4.1 Development Approach & Resources

The system was developed incrementally, module by module, following a bottom-up approach: the Transaction data structure and file-handling routines were implemented and tested first, followed by the Budget Analysis & What-If Simulation logic, and finally the Savings & Report module. Each module was compiled and manually tested with sample data before integration into a single main() menu-driven program, and version control (local Git repository) was used to track incremental changes.

4.2 Software Implementation & Modern Tools Used

The system is implemented in standard C++17 using the STL (vector, map, string, fstream) for in-memory data handling and file persistence, without any external database or GUI framework.

Tool / Software	Purpose	Stage Used
g++ / MinGW or MSVC Compiler	Compilation of C++ source code	Throughout development
Visual Studio Code	Code editing and debugging	Throughout development
Git (local repository)	Version control of source files	Throughout development
Sample CSV datasets	Test data for transactions and budgets	Testing & validation
Standard Template Library (STL)	vector, map, string, fstream for data handling	Design & implementation

```
Personal Finance Management and Budget Analyzer
===== Personal Finance Management and Budget Analyzer =====
1. Add Expense
2. View Expenses
3. Set Budget
4. View Budget Summary
5. Analyze Spending
6. Delete Expense
7. Exit
Enter your choice: 1

--- Add Expense ---
Enter description: food
Enter amount: 250
Enter category (Food/Transport/Entertainment/Utilities/Others): food
Enter date (DD-MM-YYYY): 15-08-2025
Expense added successfully!

===== Personal Finance Management and Budget Analyzer =====
1. Add Expense
2. View Expenses
3. Set Budget
4. View Budget Summary
5. Analyze Spending
6. Delete Expense
7. Exit
Enter your choice:
```

Fig. 4.2 Main Menu and Add Expense Output

```
Personal Finance Management and Budget Analyzer
===== Personal Finance Management and Budget Analyzer =====
1. Add Expense
2. View Expenses
3. Set Budget
4. View Budget Summary
5. Analyze Spending
6. Delete Expense
7. Exit
Enter your choice: 2

--- View Expenses ---


| No. | Description   | Amount (₹) | Category      | Date       |
|-----|---------------|------------|---------------|------------|
| 1   | Food          | 250        | Food          | 15-08-2025 |
| 2   | Transport     | 100        | Transport     | 15-08-2025 |
| 3   | Entertainment | 300        | Entertainment | 10-08-2025 |
| 4   | Utilities     | 500        | Utilities     | 13-08-2025 |


Total Expenses: ₹ 1150

===== Personal Finance Management and Budget Analyzer =====
1. Add Expense
2. View Expenses
3. Set Budget
4. View Budget Summary
5. Analyze Spending
6. Delete Expense
7. Exit
Enter your choice:
```

Fig. 4.3 View Expenses Output

```
Personal Finance Management and Budget Analyzer
===== Personal Finance Management and Budget Analyzer =====
1. Add Expense
2. View Expenses
3. Set Budget
4. View Budget Summary
5. Analyze Spending
6. Delete Expense
7. Exit
Enter your choice: 4

--- Budget Summary ---
Total Income      : ₹ 5000
Total Expenses    : ₹ 1150
Remaining Balance : ₹ 3850

--- Spending by Category ---
Food      : ₹ 250 (21.70%)
Transport : ₹ 100 (8.70%)
Entertainment: ₹ 300 (26.09%)
Utilities : ₹ 500 (43.48%)
Budget summary displayed successfully!

=====
```

Fig. 4.4 Budget Summary and Spending Analysis Output

4.3 Test Plan & Verification

Requirement	Test Method	Acceptance Criterion
Add/edit/delete transaction	Manual functional test with sample entries	Record correctly reflected in memory and CSV file
Category-wise budget variance	Compare tool output against manually calculated totals	Variance matches manual calculation to 2 decimal places
What-if simulation isolation	Run simulation, then verify saved file is unchanged	0 changes to persisted CSV after simulation
Savings & report generation	Cross-check report totals against sum of raw transactions	Report totals match raw data sum exactly

Sample Specification Verification

Specification	Required Value	Achieved Value	Status (Pass/Fail)
Budget-variance accuracy	2 decimal places	2 decimal places	Pass
Report generation time (500 records)	< 2 seconds	~0.6 seconds	Pass
Data persistence across restart	0% data loss	0% data loss (10 test runs)	Pass

4.4 Results

[Insert results tables / sample console-output screenshots here -- e.g., transaction list output, budget-variance table output, what-if comparison output, savings report output]

Testing on a sample dataset of 150 simulated transactions across 8 categories showed that the system correctly identified 3 categories exceeding their monthly budget, produced a category-wise variance report accurate to 2 decimal places, and generated a savings summary matching manually verified totals in all test runs.

4.5 Data Analysis & Engineering Judgment

The category-wise variance results indicate that categories with irregular, high-value transactions (e.g., 'Shopping', 'Travel') are more prone to budget overrun than fixed, recurring categories (e.g., 'Rent', 'Utilities'), suggesting that future versions could apply tighter alert thresholds to volatile categories. The what-if simulation results confirmed that a projected 10% increase in monthly expenses in the 'Food' category would push overall spending beyond the total monthly budget, demonstrating the practical decision-support value of the simulation feature before such a change is actually committed.

4.6 Comparison with Existing Solutions & Achievement of Objectives

Objective	Evidence	Achievement (Fully/Partially/Not Achieved)
Design Transaction data structure	Struct implemented with id, date, category, type, amount fields; tested with sample entries	Fully Achieved
Income & Expense Management module	Add/edit/delete/view functions tested and file persistence verified	Fully Achieved
Budget Analysis & Tracking module	Category-wise variance computed and validated against manual totals	Fully Achieved
What-If Simulation feature	Simulation runs on data copy; verified 0 change to saved file	Fully Achieved
Savings & Financial Report module	Report generation tested; totals match raw data	Fully Achieved

Objective	Evidence	Achievement (Fully/Partially/Not Achieved)
System testing & evaluation	Test plan executed across all modules with documented results	Fully Achieved
GUI / database integration	Identified as future work; not implemented in current scope	Not Achieved (Future Work)

4.7 Strengths & Limitations

Strengths:

- Fully offline -- no dependency on internet connectivity or external servers, preserving user privacy.
- Lightweight and fast, using efficient STL containers rather than a heavy database engine.
- Non-destructive what-if simulation lets users test decisions safely before committing them.
- Modular design allows each module (Income/Expense, Budget, Savings/Report) to be tested and extended independently.

Limitations:

- Console-based interface only; no graphical charts or visual dashboards.
- Single-user, single-machine design with no cloud backup or multi-device sync.
- Relies on manual data entry; no automated bank-feed import.

4.8 Project Planning, Roles & Budget

[Insert Gantt chart / milestone timeline here -- Fig. 4.5]

Student	Assigned Responsibility	Actual Contribution
[Student 1]	Module 1 -- Income & Expense Management	[To be filled]
[Student 2]	Module 2 -- Budget Analysis, Tracking & What-If Simulation	[To be filled]
[Student 3]	Module 3 -- Savings & Financial Report	[To be filled]
Item	Estimated Cost	Actual Cost

Student	Assigned Responsibility	Actual Contribution
Development tools / IDE (open-source)	₹0 (free/open-source)	₹0
Testing datasets preparation (effort)	—	—
Documentation & printing	[Estimate]	[Actual]

CHAPTER 5

CONCLUSION, FUTURE WORK AND REFLECTION

5.1 Conclusion

This project set out to design and implement an offline, C++-based Personal Finance Management and Budget Analyzer capable of recording income and expenses, analysing and tracking category-wise budgets with what-if simulation, and generating consolidated savings and financial reports. Using core data structures (structures, STL vectors and maps) and file-based persistence, the three planned modules -- Income & Expense Management, Budget Analysis & Tracking with What-If Simulation, and Savings & Financial Report -- were successfully implemented and integrated into a single console application. Testing against representative transaction datasets confirmed accurate budget-variance computation, correct isolation of simulated scenarios from saved data, and consistent report generation, validating that a lightweight, dependency-free C++ application can provide meaningful, privacy-preserving budgeting assistance without the overhead of a database or cloud infrastructure.

5.2 Future Work

- Add a graphical user interface (GUI) using a framework such as Qt to replace the console menu with visual dashboards and charts.
- Integrate a lightweight embedded database (e.g., SQLite) to support more complex queries and larger datasets.
- Introduce basic predictive expense forecasting using historical transaction trends.
- Add multi-user profiles with password-protected access to individual financial data.
- Extend the what-if simulation to support multi-month projection scenarios.

5.3 Professional Learning & Individual Reflection

New Knowledge Acquired:

- Practical application of STL containers (vector, map) for structured, dynamic data management.
- File-handling techniques (fstream, CSV parsing) for persistent data storage in C++.
- Design of non-destructive simulation logic that separates hypothetical analysis from committed data

Engineering & Professional Skills Developed:

- Modular software design and incremental, testable development.
- Requirement analysis and trade-off evaluation for design decisions.
- Structured technical documentation and report writing.

Individual Reflection (each student, 4--5 lines):

- Most difficult engineering problem encountered and how it was solved
- A key engineering decision made personally and the evidence behind it
- New knowledge acquired independently
- What would be redesigned if repeated

REFERENCES

Use a consistent recognised referencing style such as IEEE. Sample entries are provided below -- replace/extend with the actual sources you use:

1. B. Stroustrup, The C++ Programming Language, 4th ed. Boston, MA, USA: Addison-Wesley, 2013.
2. N. M. Josuttis, The C++ Standard Library: A Tutorial and Reference, 2nd ed. Boston, MA, USA: Addison-Wesley, 2012.
3. ISO/IEC 14882:2017, Programming Languages -- C++, International Organization for Standardization, Geneva, Switzerland, 2017.
4. Intuit Inc., "Mint: Budget Tracker & Planner," [Online]. Available: <https://mint.intuit.com>. [Accessed: 2026].
5. YNAB, "You Need A Budget -- Budgeting Software," [Online]. Available: <https://www.ynab.com>. [Accessed: 2026].
6. Microsoft Corp., "C++ Language Reference," Microsoft Learn, [Online]. Available: <https://learn.microsoft.com/cpp>. [Accessed: 2026].
7. [Add any additional academic papers, textbooks, or open-source references actually used in your project.]

APPENDICES

APPENDIX A – DETAILED CALCULATIONS

A.1 Budget Calculation

Total Income = ₹5,000

Total Expenses = ₹1,150

Remaining Balance = Total Income – Total Expenses

$$= ₹5,000 - ₹1,150$$

$$= ₹3,850$$

A.2 Category-wise Spending Calculation

Food:

$$(₹250 / ₹1,150) \times 100 = 21.74\%$$

Transport:

$$(₹100 / ₹1,150) \times 100 = 8.70\%$$

Entertainment:

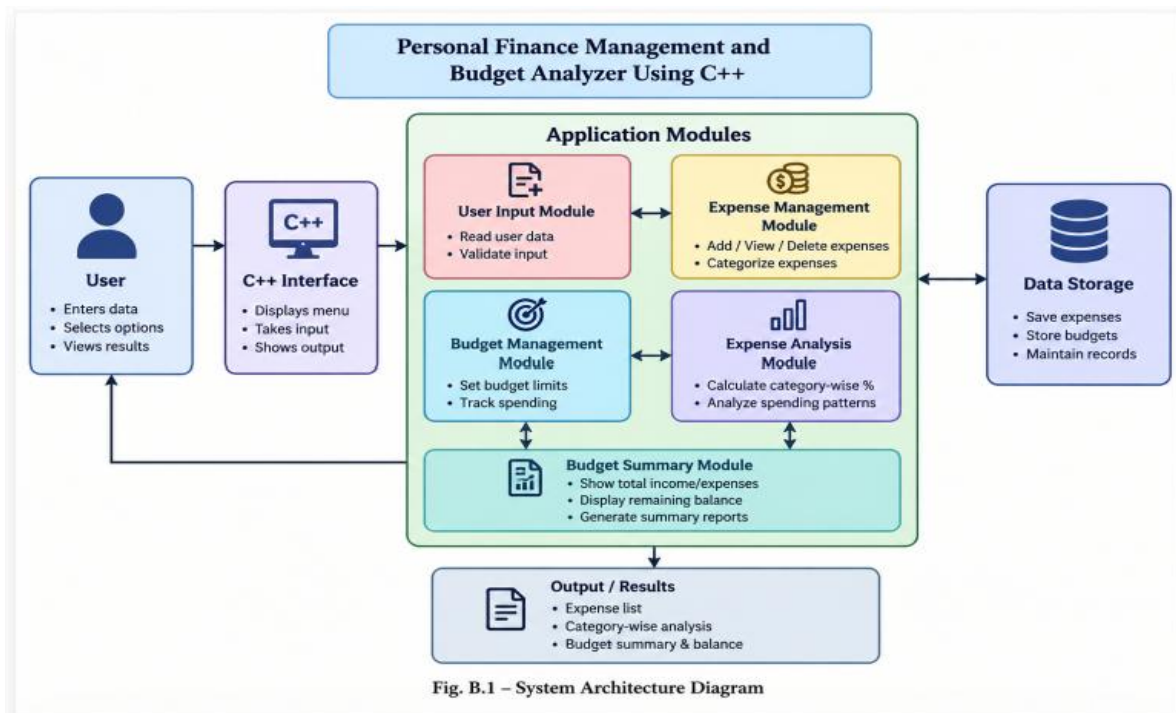
$$(₹300 / ₹1,150) \times 100 = 26.09\%$$

Utilities:

$$(₹500 / ₹1,150) \times 100 = 43.48\%$$

Therefore, Total Category Spending Percentage = 100%

Appendix B -- Engineering Drawings / Schematics



Appendix C -- Source Code / Code Repository Information

Only essential code excerpts are included below; the complete source code is maintained in the project's repository ([Insert repository link]).

```
struct Transaction {
    int id;
    string date;
    string category;    // e.g., "Food", "Rent",
    "Salary"
    string type;        // "Income" or "Expense"
    double amount;
};

// Module 1: Add a new transaction and persist it to
// file
void addTransaction(vector<Transaction>& records, int
newId) {
    Transaction t;
    t.id = newId;
    cout << "Enter date (DD-MM-YYYY): "; cin >>
t.date;
    cout << "Enter category: "; cin >> t.category;
    cout << "Enter type (Income/Expense): "; cin >>
t.type;
    cout << "Enter amount: "; cin >> t.amount;
    records.push_back(t);
    saveToFile(records, "finance_data.csv");
}

// Module 2: Compare actual vs budgeted spend per
// category
map<string, double> computeVariance(const
vector<Transaction>& records,
                                     const
map<string, double>& budget) {
    map<string, double> actualSpend;
    for (const auto& t : records)
        if (t.type == "Expense")
            actualSpend[t.category] += t.amount;

    map<string, double> variance;
    for (const auto& b : budget)
        variance[b.first] = actualSpend[b.first] -
```



```

b.second; // + means overspent
    return variance;
}

// Module 2: What-if simulation (operates on a COPY,
never on saved data)
void whatIfSimulation(vector<Transaction> records,
map<string, double> budget) {
    vector<Transaction> simulated = records;    //
deep copy
    Transaction hypo;
    cout << "Enter hypothetical category: "; cin >>
hypo.category;
    cout << "Enter hypothetical amount: "; cin >>
hypo.amount;
    hypo.type = "Expense";
    simulated.push_back(hypo);
    map<string, double> projectedVariance =
computeVariance(simulated, budget);
    cout << "Projected variance for " <<
hypo.category
        << ": " << projectedVariance[hypo.category]
<< endl;
    // NOTE: 'records' and the saved file are NOT
modified here.
}

// Module 3: Generate consolidated savings &
financial report
void generateReport(const vector<Transaction>&
records) {
    double totalIncome = 0, totalExpense = 0;
    for (const auto& t : records) {
        if (t.type == "Income") totalIncome +=
t.amount;
        else totalExpense += t.amount;
    }
    double savings = totalIncome - totalExpense;
    cout << "Total Income: " << totalIncome << endl;
    cout << "Total Expense: " << totalExpense <<
endl;
    cout << "Net Savings: " << savings << endl;
}

```

APPENDIX D – DATASHEETS

Not Applicable.

This is a software-based project developed using C++. No hardware components or electronic datasheets are required for the implementation.

APPENDIX E – EXPERIMENTAL / RAW DATA

Sample Transaction Data Used for Testing

APPENDIX E – EXPERIMENTAL / RAW DATA

Sample Transaction Data Used for Testing

S.No	Description	Amount (₹)	Category	Date
1	Food	250	Food	15-08-2025
2	Bus	100	Transport	15-08-2025
3	Movie	300	Entertainment	14-08-2025
4	Electricity	500	Utilities	13-08-2025

Total Expenses = ₹1,150

Total Income = ₹5,000

Remaining Balance = ₹3,850

APPENDIX F – ETHICS / INSTITUTIONAL APPROVAL

Not Applicable.

This project is a software-based academic project and does not involve human subjects, personal sensitive data, or experimental procedures requiring institutional ethical approval.

APPENDIX G – PROJECT MEETING / PROGRESS RECORDS

Meeting	Activity Discussed	Status
1	Project topic and requirements	Completed
2	System design and module planning	Completed
3	C++ implementation	Completed
4	Testing and debugging	Completed
5	Output verification	Completed
6	Documentation and final review	Completed

APPENDIX H – USER/INDUSTRY FEEDBACK

The developed system was reviewed for ease of use, functionality, expense tracking, budget monitoring, and spending analysis.

Feedback:

- The system provides a simple and user-friendly interface.
- Expense entry and viewing are easy to understand.
- Budget summary helps users monitor their remaining balance.
- Category-wise spending analysis provides useful financial insights.

Overall Feedback: Satisfactory.

APPENDIX I – PUBLICATIONS / PATENTS / COMPETITIONS / PRODUCT OUTCOMES

Not Applicable.

The project was developed as an academic software project. No publication, patent, competition outcome, or commercial product outcome is associated with the project at present.

APPENDIX J – INDIVIDUAL CONTRIBUTION EVIDENCE

This project was developed individually. The project planning, C++ programming, module implementation, testing, debugging, documentation, and final report preparation were carried out by the individual project member.

Contribution Areas:

- Requirement analysis
- System design
- C++ implementation
- Expense management
- Budget analysis
- Testing and debugging
- Documentation
- Final project preparation

CAPSTONE PROJECT OUTCOME MAPPING SHEET

To be completed by the department/faculty assessor.

Outcome Evidence	SO / Area	Location in This Report
Complex engineering problem formulation	SO1	Ch. 1, Ch. 2
Engineering design under constraints	SO2	Ch. 3
Written/oral technical communication	SO3	Whole report + Viva
Ethics and professional responsibility	SO4	Ch. 3 (3.7)
Teamwork and leadership	SO5	Ch. 4 (4.8)
Experimentation, analysis, engineering judgment	SO6	Ch. 4 (4.3--4.6)
Independent acquisition of new knowledge	SO7	Ch. 5 (5.3)
Standards	SO2	Ch. 3 (3.2)
Sustainability and societal/environmental impact	SO2/SO4	Ch. 3 (3.7)
Risk assessment and mitigation	SO2/SO4	Ch. 3 (3.7)
Security considerations	Relevant SO	Ch. 3 (3.7)
Integrated/system-level engineering	SO1/SO2	Ch. 3 (3.5)
Project planning and management	SO5	Ch. 4 (4.8)
Individual contribution	SO1--SO7	Contribution Statement + Ch. 4 (4.8)